

**REFACTORING PELANGGARAN LISKOV SUBSTITUTION
PRINCIPLE (LSP) DAN INTERFACE SEGREGATION
PRINCIPLE (ISP) PADA APLIKASI GUI HOSPITAL
MANAGEMENT MENGGUNAKAN PYTHON**



Disusun Oleh :

Iqbal Ghifari	(21120123140175)
Herdika Putra Devara	(21120123140112)
Muhammad Romeo Raffael	(21120123140151)

**DEPARTEMEN TEKNIK KOMPUTER
FAKULTAS TEKNIK
UNIVERSITAS DIPONEGORO
SEMARANG**

2026

KODE PROGRAM

Link Kode Github : <https://github.com/Skirnola/Tugas-PBOL-Refactor-ISP-dan-LSP.git>

Kode Sebelum Di Refactor :

hospital.py

```
from tkinter import *
import sqlite3
import tkinter
from tkinter.ttk import Separator
from tkinter import messagebox

conn = sqlite3.connect('database.db')
c = conn.cursor()
ids = []
number = []
patients = []

class BaseAppManager:

    def show_form(self):
        raise NotImplementedError

    def save_data(self):
        raise NotImplementedError

    def delete_data(self):
        raise NotImplementedError

    def generate_report(self):
        raise NotImplementedError

class Application(BaseAppManager):

    def __init__(self, window):
        self.window = window
        self.v = IntVar()
        c.execute("SELECT * FROM appointments")
        self.alldata = c.fetchall()

        self.main = Frame(window, width=450, height=400,
bg="lightblue")
        self.showdetailsframe = Frame(self.window)
        self.updateframe = Frame(self.window)
        self.deleteframe = Frame(self.window)

    def generate_report(self):
```

```

        pass

    def show_form(self):
        self.startpage()

    def save_data(self):
        self.add_appointment()

    def delete_data(self):
        self.deletee()

    def validate_input(self, val1, val2, val3, val4, val5):
        if val1 == '' or val2 == '' or val3 == '' or val4 == ''
or val5 == '':
            return False
        return True

    def startpage(self):
        self.heading = Label(self.main, text="Orchade Hospital
Appointments",
                             font=('Centaur      20      bold'),
fg='black', bg="yellow", relief=SUNKEN)
        self.heading.place(x=12, y=20)

        self.name = Label(self.main, text="Patients Name",
font=('arial 12 bold'), bg="lightblue")
        self.name.place(x=0, y=110)

        self.age = Label(self.main, text="Age", font=('arial 12
bold'), bg="lightblue")
        self.age.place(x=0, y=155)

        Label(self.main, text="Gender", font=('arial 12 bold'),
bg="lightblue").place(x=0, y=210)
        Radiobutton(self.main, text="Male", padx=20, font="ariel
10 bold",
                     variable=self.v, value=1,
bg="lightblue").place(x=130, y=210)
        Radiobutton(self.main, text="Female", padx=20,
font="ariel 10 bold",
                     variable=self.v, value=2,
bg="lightblue").place(x=220, y=210)

        self.time = Label(self.main, text="Location",
font=('arial 12 bold'), bg="lightblue")
        self.time.place(x=0, y=255)

        self.phone = Label(self.main, text="Contact Number",
font=('arial 12 bold'), bg="lightblue")

```

```

self.phone.place(x=0, y=300)

self.name_ent = Entry(self.main, width=30)
self.name_ent.place(x=140, y=115)

self.age_ent = Entry(self.main, width=30)
self.age_ent.place(x=140, y=160)

self.location_ent = Entry(self.main, width=30)
self.location_ent.place(x=140, y=258)

self.phone_ent = Entry(self.main, width=30)
self.phone_ent.place(x=140, y=310)

self.submit = Button(self.main, text="Add Appointment",
font="arial 12 bold",
width=15, height=2,
bg='lightgreen', command=self.add_appointment)
self.submit.place(x=120, y=340)

sql2 = "SELECT id FROM appointments"
self.result = c.execute(sql2)
for self.row in self.result:
    self.id = self.row[0]
    ids.append(self.id)

self.new = sorted(ids)
self.final_id = self.new[len(ids) - 1]

self.logs = Label(self.main, text="Total\n Appointments",
font=('arial 10 bold'), fg='black',
bg="lightblue")
self.logs.place(x=340, y=320)
Label(self.main, text=" " + str(self.final_id),
width=8, height=1, relief=SUNKEN).place(x=360,
y=360)

self.main.pack()

def add_appointment(self):
    self.val1 = self.name_ent.get()
    self.val2 = self.age_ent.get()
    if self.v.get() == 1:
        self.val3 = "Male"
    elif self.v.get() == 2:
        self.val3 = "Female"
    else:
        self.val3 = "Not Specified"
    self.val4 = self.location_ent.get()

```

```

        self.val5 = self.phone_ent.get()

        if not self.validate_input(self.val1, self.val2,
self.val3, self.val4, self.val5):
            tkinter.messagebox.showinfo("Warning", "Please Fill
Up All Boxes")
        else:
            sql = "INSERT INTO 'appointments' (name, age, gender,
location, phone) VALUES(?,?,?, ?, ?)"
            c.execute(sql, (self.val1, self.val2, self.val3,
self.val4, self.val5))
            conn.commit()
            tkinter.messagebox.showinfo("Success",
"\n Appointment for " +
str(self.val1) + " has been created")
            self.main.destroy()
            self.__init__(self.window)
            self.startpage()

    def homee(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)
        self.startpage()
        self.main.pack()

    def showdetails(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)
        count1 = 0
        count2 = 0
        clmnname = ['App no', 'name', 'age', 'gender',
'location', 'contact no']
        for i in range(len(clmnname)):
            Label(self.showdetailsframe, text=clmnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)
            Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
        for i in range(len(clmnname)):
            Label(self.showdetailsframe, text=clmnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)
            Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
        for i in range(len(self.alldata)):

```

```

        for j in range(6):
            Label(self.showdetailsframe,
text=self.alldata[i][j], font="ariel 10").grid(
                row=count1 + 2, column=count2 * 2)
            Separator(self.showdetailsframe,
orient=VERTICAL).grid(
                row=count1 + 2, column=count2 * 2 + 1,
sticky='ns')
            count2 += 1
        count2 = 0
        count1 += 1
        self.showdetailsframe.pack()

def updatee(self):
    self.main.destroy()
    self.showdetailsframe.destroy()
    self.updateframe.destroy()
    self.deleteframe.destroy()
    self.__init__(self.window)

    self.id = Label(self.updateframe, text="Search Appoinment
Number To Update",
                    font=('arial 12 bold'), fg="red")
    self.id.place(x=0, y=12)
    self.idnet = Entry(self.updateframe, width=10)
    self.idnet.place(x=320, y=18)
    self.search = Button(self.updateframe, text="Search",
font="aried 12 bold",
                    width=10, height=1,
bg='lightgreen', command=self.update1)
    self.search.place(x=160, y=50)
    self.updateframe.pack(fill='both', expand=True)

def update1(self):
    self.input = self.idnet.get()
    sql = "SELECT * FROM appointments WHERE id LIKE ?"
    self.res = c.execute(sql, (self.input,))
    for self.row in self.res:
        self.name1 = self.row[1]
        self.age = self.row[2]
        self.gender = self.row[3]
        self.location = self.row[4]
        self.phone = self.row[5]

    Label(self.updateframe, text="Patient's Name",
font=('arial 14 bold')).place(x=0, y=140)
    Label(self.updateframe, text="Age", font=('arial
14 bold')).place(x=0, y=180)

```

```

        Label(self.updateframe, text="Gender",
font=('arial 14 bold')).place(x=0, y=220)
        Label(self.updateframe, text="Location",
font=('arial 14 bold')).place(x=0, y=260)
        Label(self.updateframe, text="Phone Number",
font=('arial 14 bold')).place(x=0, y=300)

        self.ent1 = Entry(self.updateframe, width=30);
self.ent1.place(x=180, y=140)
        self.ent1.insert(END, str(self.name1))
        self.ent2 = Entry(self.updateframe, width=30);
self.ent2.place(x=180, y=180)
        self.ent2.insert(END, str(self.age))
        self.ent3 = Entry(self.updateframe, width=30);
self.ent3.place(x=180, y=220)
        self.ent3.insert(END, str(self.gender))
        self.ent4 = Entry(self.updateframe, width=30);
self.ent4.place(x=180, y=260)
        self.ent4.insert(END, str(self.location))
        self.ent5 = Entry(self.updateframe, width=30);
self.ent5.place(x=180, y=300)
        self.ent5.insert(END, str(self.phone))

        Button(self.updateframe, text="Update", font="aried 12
bold",
width=10, height=1, bg='lightgreen',
command=self.update2).place(x=25, y=340)
        self.updateframe.pack()

    def update2(self):
        query = "UPDATE appointments SET name=?, age=?, gender=?,
location=?, phone=? WHERE id LIKE ?"
        c.execute(query, (self.ent1.get(), self.ent2.get(),
self.ent3.get(),
self.ent4.get(), self.ent5.get(),
self.idnet.get()))
        conn.commit()
        tkinter.messagebox.showinfo("Updated", "Successfully
Updated.")
        self.updateframe.destroy()
        self.__init__(self.window)
        self.updatee()
        self.updateframe.pack()

    def deletee(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()

```

```

        self.__init__(self.window)

        self.id = Label(self.deleteframe, text="Search Appointment
Number To Delete",
                        font=('arial 12 bold'), fg="red")
        self.id.place(x=0, y=12)
        self.idnet = Entry(self.deleteframe, width=10)
        self.idnet.place(x=320, y=18)
        self.search = Button(self.deleteframe, text="Search",
font="aried 12 bold",
                        width=10, height=1,
bg='lightgreen', command=self.deletel)
        self.search.place(x=160, y=50)
        self.deleteframe.pack(fill='both', expand=True)

    def deletel(self):
        self.input = self.idnet.get()
        sql = "SELECT * FROM appointments WHERE id LIKE ?"
        self.res = c.execute(sql, (self.input,))
        for self.row in self.res:
            self.name1 = self.row[1]
            self.age = self.row[2]
            self.gender = self.row[3]
            self.location = self.row[4]
            self.phone = self.row[5]

        Label(self.deleteframe, text="Patient's Name",
font=('arial 14 bold')).place(x=0, y=140)
        Label(self.deleteframe, text="Age", font=('arial
14 bold')).place(x=0, y=180)
        Label(self.deleteframe, text="Gender",
font=('arial 14 bold')).place(x=0, y=220)
        Label(self.deleteframe, text="Location",
font=('arial 14 bold')).place(x=0, y=260)
        Label(self.deleteframe, text="Phone Number",
font=('arial 14 bold')).place(x=0, y=300)

        self.ent1 = Entry(self.deleteframe, width=30);
self.ent1.place(x=180, y=140)
        self.ent1.insert(END, str(self.name1))
        self.ent2 = Entry(self.deleteframe, width=30);
self.ent2.place(x=180, y=180)
        self.ent2.insert(END, str(self.age))
        self.ent3 = Entry(self.deleteframe, width=30);
self.ent3.place(x=180, y=220)
        self.ent3.insert(END, str(self.gender))
        self.ent4 = Entry(self.deleteframe, width=30);
self.ent4.place(x=180, y=260)
        self.ent4.insert(END, str(self.location))

```



```

        self.ent5 = Entry(self.deleteframe, width=30);
self.ent5.place(x=180, y=300)
        self.ent5.insert(END, str(self.phone))

        Button(self.deleteframe, text="Delete", font="ariel 12
bold",
                width=10, height=1, bg='lightgreen',
command=self.delete2).place(x=25, y=340)
        self.deleteframe.pack()

    def delete2(self):
        sql2 = "DELETE FROM appointments WHERE id LIKE ?"
        c.execute(sql2, (self.idnet.get(),))
        conn.commit()
        tkinter.messagebox.showinfo("Success", "Deleted
Successfully")
        self.ent1.destroy(); self.ent2.destroy();
self.ent3.destroy()
        self.ent4.destroy(); self.ent5.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)
        self.deletee()
        self.deleteframe.pack()

class AdminApplication(Application):

    def validate_input(self, val1, val2, val3, val4, val5):
        if val1 == '' or val2 == '' or val3 == '' or val4 == ''
or val5 == '':
            raise ValueError(
                "Admin mode: semua field wajib diisi, tidak boleh
kosong!"
            )
        return True

class ReportOnlyApp(Application):

    def save_data(self):
        pass

    def delete_data(self):
        pass

    def showdetails(self):
        raise NotImplementedError(
            "ReportOnlyApp tidak mendukung tampilan tabel GUI. "
            "Gunakan generate_report() sebagai gantinya."
        )

```

```

def generate_report(self):
    print("=== LAPORAN APPOINTMENTS ===")
    for row in self.alldata:
        print(row)

def menubar():
    main_menu = Menu()
    window.config(menu=main_menu)
    file_menu = Menu(main_menu, tearoff=False)
    main_menu.add_cascade(label="Menu", menu=file_menu)
    file_menu.add_command(label="Home", command=b.homeee)
    file_menu.add_command(label="Show details",
command=b.showdetails)
    file_menu.add_command(label="Update",
command=b.updateee)
    file_menu.add_command(label="Delete",
command=b.deleteee)
    file_menu.add_separator()
    file_menu.add_command(label="Exit",
command=window.quit)

window = Tk()
b = Application(window)
b.startpage()
window.config(menu=menubar())
window.title("Hospital Management")
window.iconbitmap(r'medkit.ico')
window.geometry("450x400")
window.resizable(False, False)
window.mainloop()

```

Sesudah Di Refactor :

hospital.py

```

from tkinter import *
import sqlite3
import tkinter
from tkinter.ttk import Separator
from tkinter import messagebox

conn = sqlite3.connect('database.db')
c = conn.cursor()
ids = []
number = []
patients = []

class IFormView:

```

```

    def show_form(self):
        raise NotImplementedError

class IDataWriter:
    def save_data(self):
        raise NotImplementedError

    def delete_data(self):
        raise NotImplementedError

class IReportable:
    def generate_report(self):
        raise NotImplementedError

class Application(IFormView, IDataWriter):

    def __init__(self, window):
        self.window = window
        self.v = IntVar()
        c.execute("SELECT * FROM appointments")
        self.alldata = c.fetchall()

        self.main = Frame(window, width=450, height=400,
bg="lightblue")
        self.showdetailsframe = Frame(self.window)
        self.updateframe = Frame(self.window)
        self.deleteframe = Frame(self.window)

    def show_form(self):
        self.startpage()

    def save_data(self):
        self.add_appointment()

    def delete_data(self):
        self.deletee()

    def validate_input(self, val1, val2, val3, val4, val5):
        if val1 == '' or val2 == '' or val3 == '' or val4 == ''
or val5 == '':
            return False
        return True

    def startpage(self):
        self.heading = Label(self.main, text="Orchade Hospital
Appointments",

```

```

                                font=('Centaur 20 bold'),
fg='black', bg="yellow", relief=SUNKEN)
    self.heading.place(x=12, y=20)

    self.name = Label(self.main, text="Patients Name",
font=('arial 12 bold'), bg="lightblue")
    self.name.place(x=0, y=110)

    self.age = Label(self.main, text="Age", font=('arial 12
bold'), bg="lightblue")
    self.age.place(x=0, y=155)

    Label(self.main, text="Gender", font=('arial 12 bold'),
bg="lightblue").place(x=0, y=210)
    Radiobutton(self.main, text="Male", padx=20, font="ariel
10 bold",
                                variable=self.v, value=1,
bg="lightblue").place(x=130, y=210)
    Radiobutton(self.main, text="Female", padx=20,
font="ariel 10 bold",
                                variable=self.v, value=2,
bg="lightblue").place(x=220, y=210)

    self.time = Label(self.main, text="Location",
font=('arial 12 bold'), bg="lightblue")
    self.time.place(x=0, y=255)

    self.phone = Label(self.main, text="Contact Number",
font=('arial 12 bold'), bg="lightblue")
    self.phone.place(x=0, y=300)

    self.name_ent = Entry(self.main, width=30)
    self.name_ent.place(x=140, y=115)

    self.age_ent = Entry(self.main, width=30)
    self.age_ent.place(x=140, y=160)

    self.location_ent = Entry(self.main, width=30)
    self.location_ent.place(x=140, y=258)

    self.phone_ent = Entry(self.main, width=30)
    self.phone_ent.place(x=140, y=310)

    self.submit = Button(self.main, text="Add Appointment",
font="ariel 12 bold",
                                width=15, height=2,
bg='lightgreen', command=self.add_appointment)
    self.submit.place(x=120, y=340)

```

```

        sql2 = "SELECT id FROM appointments"
        self.result = c.execute(sql2)
        for self.row in self.result:
            self.id = self.row[0]
            ids.append(self.id)

        self.new = sorted(ids)
        self.final_id = self.new[len(ids) - 1]

        self.logs = Label(self.main, text="Total\n
Appointments",
                           font=('arial 10 bold'), fg='black',
bg="lightblue")
        self.logs.place(x=340, y=320)
        Label(self.main, text=" " + str(self.final_id),
               width=8, height=1, relief=SUNKEN).place(x=360,
y=360)

        self.main.pack()

    def add_appointment(self):
        self.val1 = self.name_ent.get()
        self.val2 = self.age_ent.get()
        if self.v.get() == 1:
            self.val3 = "Male"
        elif self.v.get() == 2:
            self.val3 = "Female"
        else:
            self.val3 = "Not Specified"
        self.val4 = self.location_ent.get()
        self.val5 = self.phone_ent.get()

        if not self.validate_input(self.val1, self.val2,
self.val3, self.val4, self.val5):
            tkinter.messagebox.showinfo("Warning", "Please Fill
Up All Boxes")
        else:
            sql = "INSERT INTO 'appointments' (name, age,
gender, location, phone) VALUES(?,?,?,?,?)"
            c.execute(sql, (self.val1, self.val2, self.val3,
self.val4, self.val5))
            conn.commit()
            tkinter.messagebox.showinfo("Success",
"\n Appointment for " +
str(self.val1) + " has been created")
            self.main.destroy()
            self.__init__(self.window)
            self.startpage()

```

```

def homee(self):
    self.main.destroy()
    self.showdetailsframe.destroy()
    self.updateframe.destroy()
    self.deleteframe.destroy()
    self.__init__(self.window)
    self.startpage()
    self.main.pack()

def showdetails(self):
    self.main.destroy()
    self.showdetailsframe.destroy()
    self.updateframe.destroy()
    self.deleteframe.destroy()
    self.__init__(self.window)
    count1 = 0
    count2 = 0
    clmnname = ['App no', 'name', 'age', 'gender',
'location', 'contact no']
    for i in range(len(clmnname)):
        Label(self.showdetailsframe, text=clmnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)
        Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
    for i in range(len(clmnname)):
        Label(self.showdetailsframe, text=clmnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)
        Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
    for i in range(len(self.alldata)):
        for j in range(6):
            Label(self.showdetailsframe,
text=self.alldata[i][j], font="ariel 10").grid(
                row=count1 + 2, column=count2 * 2)
            Separator(self.showdetailsframe,
orient=VERTICAL).grid(
                row=count1 + 2, column=count2 * 2 + 1,
sticky='ns')
            count2 += 1
        count2 = 0
        count1 += 1
    self.showdetailsframe.pack()

def updatee(self):
    self.main.destroy()
    self.showdetailsframe.destroy()
    self.updateframe.destroy()
    self.deleteframe.destroy()
    self.__init__(self.window)

```

```

        self.id = Label(self.updateframe, text="Search
Appointment Number To Update",
                        font=('arial 12 bold'), fg="red")
        self.id.place(x=0, y=12)
        self.idnet = Entry(self.updateframe, width=10)
        self.idnet.place(x=320, y=18)
        self.search = Button(self.updateframe, text="Search",
font="aried 12 bold",
                                width=10, height=1,
bg='lightgreen', command=self.update1)
        self.search.place(x=160, y=50)
        self.updateframe.pack(fill='both', expand=True)

    def update1(self):
        self.input = self.idnet.get()
        sql = "SELECT * FROM appointments WHERE id LIKE ?"
        self.res = c.execute(sql, (self.input,))
        for self.row in self.res:
            self.name1 = self.row[1]
            self.age = self.row[2]
            self.gender = self.row[3]
            self.location = self.row[4]
            self.phone = self.row[5]

        Label(self.updateframe, text="Patient's Name",
font=('arial 14 bold')).place(x=0, y=140)
        Label(self.updateframe, text="Age",
font=('arial 14 bold')).place(x=0, y=180)
        Label(self.updateframe, text="Gender",
font=('arial 14 bold')).place(x=0, y=220)
        Label(self.updateframe, text="Location",
font=('arial 14 bold')).place(x=0, y=260)
        Label(self.updateframe, text="Phone Number",
font=('arial 14 bold')).place(x=0, y=300)

        self.ent1 = Entry(self.updateframe, width=30);
self.ent1.place(x=180, y=140)
        self.ent1.insert(END, str(self.name1))
        self.ent2 = Entry(self.updateframe, width=30);
self.ent2.place(x=180, y=180)
        self.ent2.insert(END, str(self.age))
        self.ent3 = Entry(self.updateframe, width=30);
self.ent3.place(x=180, y=220)
        self.ent3.insert(END, str(self.gender))
        self.ent4 = Entry(self.updateframe, width=30);
self.ent4.place(x=180, y=260)
        self.ent4.insert(END, str(self.location))

```

```

        self.ent5 = Entry(self.updateframe, width=30);
self.ent5.place(x=180, y=300)
        self.ent5.insert(END, str(self.phone))

        Button(self.updateframe, text="Update", font="arial 12
bold",
                width=10, height=1, bg='lightgreen',
command=self.update2).place(x=25, y=340)
        self.updateframe.pack()

    def update2(self):
        query = "UPDATE appointments SET name=?, age=?,
gender=?, location=?, phone=? WHERE id LIKE ?"
        c.execute(query, (self.ent1.get(), self.ent2.get(),
self.ent3.get(),
                                self.ent4.get(), self.ent5.get(),
self.idnet.get()))
        conn.commit()
        tkinter.messagebox.showinfo("Updated", "Successfully
Updated.")
        self.updateframe.destroy()
        self.__init__(self.window)
        self.updatee()
        self.updateframe.pack()

    def deletee(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)

        self.id = Label(self.deleteframe, text="Search
Appointment Number To Delete",
                        font=('arial 12 bold'), fg="red")
        self.id.place(x=0, y=12)
        self.idnet = Entry(self.deleteframe, width=10)
        self.idnet.place(x=320, y=18)
        self.search = Button(self.deleteframe, text="Search",
font="arial 12 bold",
                                width=10, height=1,
bg='lightgreen', command=self.delete1)
        self.search.place(x=160, y=50)
        self.deleteframe.pack(fill='both', expand=True)

    def delete1(self):
        self.input = self.idnet.get()
        sql = "SELECT * FROM appointments WHERE id LIKE ?"
        self.res = c.execute(sql, (self.input,))

```



```

        for self.row in self.res:
            self.name1      = self.row[1]
            self.age        = self.row[2]
            self.gender     = self.row[3]
            self.location   = self.row[4]
            self.phone      = self.row[5]

            Label(self.deleteframe, text="Patient's Name",
font=('arial 14 bold')).place(x=0, y=140)
            Label(self.deleteframe, text="Age",
font=('arial 14 bold')).place(x=0, y=180)
            Label(self.deleteframe, text="Gender",
font=('arial 14 bold')).place(x=0, y=220)
            Label(self.deleteframe, text="Location",
font=('arial 14 bold')).place(x=0, y=260)
            Label(self.deleteframe, text="Phone Number",
font=('arial 14 bold')).place(x=0, y=300)

            self.ent1 = Entry(self.deleteframe, width=30);
self.ent1.place(x=180, y=140)
            self.ent1.insert(END, str(self.name1))
            self.ent2 = Entry(self.deleteframe, width=30);
self.ent2.place(x=180, y=180)
            self.ent2.insert(END, str(self.age))
            self.ent3 = Entry(self.deleteframe, width=30);
self.ent3.place(x=180, y=220)
            self.ent3.insert(END, str(self.gender))
            self.ent4 = Entry(self.deleteframe, width=30);
self.ent4.place(x=180, y=260)
            self.ent4.insert(END, str(self.location))
            self.ent5 = Entry(self.deleteframe, width=30);
self.ent5.place(x=180, y=300)
            self.ent5.insert(END, str(self.phone))

            Button(self.deleteframe, text="Delete", font="aried 12
bold",
                    width=10, height=1, bg='lightgreen',
command=self.delete2).place(x=25, y=340)
            self.deleteframe.pack()

        def delete2(self):
            sql2 = "DELETE FROM appointments WHERE id LIKE ?"
            c.execute(sql2, (self.idnet.get(),))
            conn.commit()
            tkinter.messagebox.showinfo("Success", "Deleted
Successfully")
            self.ent1.destroy(); self.ent2.destroy();
self.ent3.destroy()
            self.ent4.destroy(); self.ent5.destroy()

```

```

        self.deleteframe.destroy()
        self.__init__(self.window)
        self.deletee()
        self.deleteframe.pack()

class AdminApplication(Application):

    def validate_input(self, val1, val2, val3, val4, val5):
        if val1 == '' or val2 == '' or val3 == '' or val4 == ''
or val5 == '':
            return False
        return True

class ReportOnlyApp(IFormView, IReportable):

    def __init__(self, window):
        self.window = window
        self.v = IntVar()
        c.execute("SELECT * FROM appointments")
        self.alldata = c.fetchall()

        self.main = Frame(window, width=450, height=400,
bg="lightblue")
        self.showdetailsframe = Frame(self.window)
        self.updateframe = Frame(self.window)
        self.deleteframe = Frame(self.window)

    def show_form(self):
        self.startpage()

    def generate_report(self):
        print("=== LAPORAN APPOINTMENTS ===")
        for row in self.alldata:
            print(row)

    def showdetails(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)
        count1 = 0
        count2 = 0
        clmnname = ['App no', 'name', 'age', 'gender',
'location', 'contact no']
        for i in range(len(clmnname)):
            Label(self.showdetailsframe, text=clmnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)

```

```

        Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
        for i in range(len(clmnnname)):
            Label(self.showdetailsframe, text=clmnnname[i],
font="ariel 12 bold").grid(row=0, column=i * 2)
            Separator(self.showdetailsframe,
orient=VERTICAL).grid(row=0, column=i * 2 + 1, sticky='ns')
            for i in range(len(self.alldata)):
                for j in range(6):
                    Label(self.showdetailsframe,
text=self.alldata[i][j], font="ariel 10").grid(
                        row=count1 + 2, column=count2 * 2)
                    Separator(self.showdetailsframe,
orient=VERTICAL).grid(
                        row=count1 + 2, column=count2 * 2 + 1,
sticky='ns')
                    count2 += 1
                count2 = 0
                count1 += 1
            self.showdetailsframe.pack()

    def startpage(self):
        self.heading = Label(self.main, text="Orchade Hospital
Appointments",
                                font=('Centaur 20 bold'),
fg='black', bg="yellow", relief=SUNKEN)
        self.heading.place(x=12, y=20)

        self.name = Label(self.main, text="Patients Name",
font=('arial 12 bold'), bg="lightblue")
        self.name.place(x=0, y=110)

        self.age = Label(self.main, text="Age", font=('arial 12
bold'), bg="lightblue")
        self.age.place(x=0, y=155)

        Label(self.main, text="Gender", font=('arial 12 bold'),
bg="lightblue").place(x=0, y=210)
        Radiobutton(self.main, text="Male", padx=20, font="ariel
10 bold",
                                variable=self.v, value=1,
bg="lightblue").place(x=130, y=210)
        Radiobutton(self.main, text="Female", padx=20,
font="ariel 10 bold",
                                variable=self.v, value=2,
bg="lightblue").place(x=220, y=210)

        self.time = Label(self.main, text="Location",
font=('arial 12 bold'), bg="lightblue")

```

```

        self.time.place(x=0, y=255)

        self.phone = Label(self.main, text="Contact Number",
font=('arial 12 bold'), bg="lightblue")
        self.phone.place(x=0, y=300)

        self.name_ent = Entry(self.main, width=30)
        self.name_ent.place(x=140, y=115)

        self.age_ent = Entry(self.main, width=30)
        self.age_ent.place(x=140, y=160)

        self.location_ent = Entry(self.main, width=30)
        self.location_ent.place(x=140, y=258)

        self.phone_ent = Entry(self.main, width=30)
        self.phone_ent.place(x=140, y=310)

        sql2 = "SELECT id FROM appointments"
        self.result = c.execute(sql2)
        for self.row in self.result:
            self.id = self.row[0]
            ids.append(self.id)

        self.new = sorted(ids)
        self.final_id = self.new[len(ids) - 1]

        self.logs = Label(self.main, text="Total\n
Appointments",
                        font=('arial 10 bold'), fg='black',
bg="lightblue")
        self.logs.place(x=340, y=320)
        Label(self.main, text=" " + str(self.final_id),
            width=8, height=1, relief=SUNKEN).place(x=360,
y=360)

        self.main.pack()

    def homee(self):
        self.main.destroy()
        self.showdetailsframe.destroy()
        self.updateframe.destroy()
        self.deleteframe.destroy()
        self.__init__(self.window)
        self.startpage()
        self.main.pack()

    def updatee(self):
        pass

```

```

        def deletee(self):
            pass

def menubar():
    main_menu = Menu()
    window.config(menu=main_menu)
    file_menu = Menu(main_menu, tearoff=False)
    main_menu.add_cascade(label="Menu", menu=file_menu)
    file_menu.add_command(label="Home", command=b.homee)
    file_menu.add_command(label="Show details",
command=b.showdetails)
    file_menu.add_command(label="Update",
command=b.updatee)
    file_menu.add_command(label="Delete",
command=b.deletee)
    file_menu.add_separator()
    file_menu.add_command(label="Exit",
command=window.quit)

window = Tk()
b = Application(window)
b.startpage()
window.config(menu=menubar())
window.title("Hospital Management")
window.iconbitmap(r'medkit.ico')
window.geometry("450x400")
window.resizable(False, False)
window.mainloop()

```

Tampilan GUI Aplikasi:

Hospital Management

Menu

Orchade Hospital Appointments

Patients Name

Romeo

Age

67

Gender

☒ Male

☐ Female

Location

Semarang

Contact Number

082947365

Add Appointment

Total Appointments

6

KONTRIBUSI SETIAP ANGGOTA KELOMPOK

1. Iqbal Ghifari – 21120123140175

Case ISP Fat interface, client dipaksa bergantung pada method yang tidak dipakai.

Kode Sebelum di Refactor :

```
You, 5 hours ago | 1 author (You)
class BaseAppManager:

    def show_form(self):
        raise NotImplementedError

    def save_data(self):
        raise NotImplementedError

    def delete_data(self):
        raise NotImplementedError

    def generate_report(self):
        raise NotImplementedError

You, 4 hours ago | 1 author (You)
class Application(BaseAppManager):

    def __init__(self, window):
        self.window = window
        self.v = IntVar()
        c.execute("SELECT * FROM appointments")
        self.alldata = c.fetchall()

        self.main = Frame(window, width=450, height=400, bg="lightblue")
        self.showdetailsframe = Frame(self.window)
        self.updateframe = Frame(self.window)
        self.deleteframe = Frame(self.window)

    def generate_report(self):
        pass

    def show_form(self):
        self.startpage()
```

Semua subclass dipaksa untuk mengimplementasi keempat method tersebut.

Padahal kebutuhan tiap subclass berbeda:

- Application : butuh show_form, save_data, delete_data tidak butuh generate_report.
- ReportOnlyApp : butuh show_form, generate_report tidak butuh save_data, delete_data.

Bisa dilihat pada bagian

```
class Application(BaseAppManager):
    def generate_report(self):
        pass # ← terpaksa ada, tidak pernah dipakai
```

Karena mewarisi BaseAppManager, kelas Application dipaksa mengimplementasi generate_report() meskipun method tersebut sama sekali tidak dibutuhkan oleh Application.

```
class ReportOnlyApp(Application):
    def save_data(self):
        pass
    def delete_data(self):
        pass
    def generate_report(self):
        print('=== LAPORAN ===')
        for row in self.alldata:
            print(row)
```

Sebaliknya, kelas ReportOnlyApp yang hanya bertugas menampilkan laporan, dipaksa mengimplementasi save_data() dan delete_data() yang tidak relevan sama sekali.

Pelanggaran Masalah :

1. Application dipaksa implementasi generate_report(). Application adalah kelas utama untuk mengelola appointment pasien.

Tugasnya: menampilkan form, menyimpan data, dan menghapus data. Namun karena BaseAppManager memiliki generate_report(), Application terpaksa mengimplementasinya dengan isi 'pass' yang tidak berguna.

Dampak: jika generate_report() dipanggil pada objek Application, tidak ada error tapi juga tidak ada output - perilaku yang membingungkan.

2. ReportOnlyApp dipaksa implementasi save_data() & delete_data(). ReportOnlyApp dirancang HANYA untuk menampilkan laporan. Ia tidak seharusnya menyimpan atau menghapus data apapun. Namun karena mewarisi BaseAppManager, ia terpaksa mengimplementasi save_data() dan delete_data() dengan isi 'pass' yang tidak relevan.

Dampak: kode menjadi menyesatkan — ada method simpan & hapus di kelas yang sebenarnya tidak boleh melakukan operasi tersebut.

Kode Setelah di Refactor :


```

You, 6 hours ago | 1 author (You)
class IFormView:
    def show_form(self):
        raise NotImplementedError

You, 6 hours ago | 1 author (You)
class IDataWriter:
    def save_data(self):
        raise NotImplementedError

    def delete_data(self):
        raise NotImplementedError

You, 6 hours ago | 1 author (You)
class IReportable:
    def generate_report(self):
        raise NotImplementedError

You, 6 hours ago | 1 author (You)
class Application(IFormView, IDataWriter):

    def __init__(self, window):
        self.window = window
        self.v = IntVar()
        c.execute("SELECT * FROM appointments")
        self.alldata = c.fetchall()

        self.main = Frame(window, width=450, height=400, bg="lightblue")
        self.showdetailsframe = Frame(self.window)
        self.updateframe = Frame(self.window)
        self.deleteframe = Frame(self.window)

    def show_form(self):
        self.startpage()

    def save_data(self):
        self.add_appointment()

    def delete_data(self):
        self.deletee()

```

Solusi ISP adalah memecah BaseAppManager menjadi tiga interface yang lebih kecil dan spesifik, masing-masing hanya berisi method yang benar-benar relevan dengan satu tanggung jawab. Application sekarang hanya mewarisi IFormView dan IDataWriter, sama seperti dua interface yang ia butuhkan. Method generate_report() yang sebelumnya ada sebagai dummy pass kini sudah hilang sepenuhnya. ReportOnlyApp kini hanya mewarisi IFormView dan IReportable. Method save_data() dan delete_data() yang sebelumnya ada sebagai dummy pass kini sudah hilang sepenuhnya.

Jika dibandingkan maka seperti ini :

Aspek	Sebelum Refactor	Sesudah Refactor
Interface	1 fat interface (BaseAppManager) 4 method sekaligus	3 interface kecil (IFormView, IDataWriter, IReportable)

Application	Terpaksa punya generate_report(): pass yang tidak dipakai	Hanya warisi IFormView dan IDataWriter, tidak ada dummy method
ReportOnlyApp	Terpaksa punya save_data(): pass delete_data(): pass	Hanya warisi IFormView dan IReportable Tidak ada dummy method
Coupling	Tinggi, perubahan pada generate_report() mempengaruhi Application	Rendah, perubahan pada IReportable tidak mempengaruhi Application
Keterbacaan	Membingungkan, ada method yang tidak jelas tujuannya	Jelas, setiap kelas hanya punya method yang relevan

2. Herdika Putra Devara – 21120123140112

Kode Sebelum di Refactor :

```
class Application(BaseAppManager):
    def validate_input(self, val1, val2, val3, val4,
val5):
        if val1 == '' or val2 == '' or val3 == '' or
val4 == '' or val5 == '':
            return False
        return True

class AdminApplication(Application):
    def validate_input(self, val1, val2, val3, val4,
val5):
        if val1 == '' or val2 == '' or val3 == '' or
val4 == '' or val5 == '':
            raise ValueError(
                "Admin mode: semua field wajib
diisi, tidak boleh kosong!"
```

```
)  
    return True
```

Pada kode `Application.validate_input()` membuat kontrak yang jelas kepada pemanggilnya, yaitu method ini selalu mengembalikan nilai `bool - True` jika semua field terisi, dan `False` jika ada yang kosong. Kode pemanggil pada `add_appointment()` bergantung pada kontrak tersebut dengan menulis `if not self.validate_input(...)`.

`AdminApplication` sebagai subclass mengubah perilaku metode tersebut. Ketika field ada yang kosong, alih - alih mengembalikan `False` seperti yang dilakukan parent . `AdminApplication.validate_input()` memberikan `ValueError` yang merupakan pelanggaran LSP karena subclass memperketat kontrak parent dari yang seharusnya mengembalikan `False` malah melempar exception.

Kode Sesudah di Refactor :

```
class Application(IFormView, IDataWriter):  
    def validate_input(self, val1, val2, val3, val4, val5):  
        if val1 == '' or val2 == '' or val3 == '' or val4  
        == '' or val5 == '':  
            return False  
        return True  
class AdminApplication(Application):  
    def validate_input(self, val1, val2, val3, val4, val5):  
        if val1 == '' or val2 == '' or val3 == '' or val4  
        == '' or val5 == '':  
            return False
```

Pada kode sudah di refactoring, `AdminApplication.validate_input()` dikembalikan ke perilaku yang konsisten dengan parent, yaitu cukup mengembalikan `False` ketika ada field yang kosong dan `True` ketika semua field terisi. Dengan begitu, `AdminApplication` kini benar-benar bisa menggantikan `Application` di mana saja tanpa menimbulkan perilaku tak terduga, sehingga LSP pun terpenuhi.

3. Muhammad Romeo Raffael - 21120123140151

Pada sistem *Orchade Hospital Appointments*, kelas `ReportOnlyApp` memiliki peran tunggal yaitu untuk menangani dan menampilkan laporan data. Namun, pada implementasi awalnya, kelas ini melanggar dua prinsip SOLID sekaligus, yaitu ISP dan LSP.

Kelas `ReportOnlyApp` mewarisi `fat` interface (`BaseAppManager/Application`) yang memaksanya untuk bergantung pada method yang sama sekali tidak dibutuhkan. Karena pewarisan ini, `ReportOnlyApp` terpaksa mengimplementasikan dummy method yang isinya hanya `pass`.

Kode Pelanggaran ISP yang belum diperbaiki

```
class ReportOnlyApp(Application):
    def save_data(self):
        pass # Dummy method, tidak relevan dengan ReportOnlyApp

    def delete_data(self):
        pass # Dummy method, tidak relevan dengan ReportOnlyApp
```

Kelas parent (`Application`) memiliki *method* `showdetails()` dengan kontrak bahwa *method* tersebut bertugas menampilkan tabel data ke GUI dan tidak pernah melakukan *raise exception*.

Namun, subclass `ReportOnlyApp` mengubah sifat ini dengan melakukan *override* pada `showdetails()` dan memaksa program menghasilkan *error* (*raise NotImplementedError*). Akibatnya, jika objek `Application` diganti dengan `ReportOnlyApp`, program akan *crash*, yang mana merupakan pelanggaran langsung terhadap LSP.

Kode Pelanggaran LSP yang belum diperbaiki

```
class ReportOnlyApp(Application):
    def showdetails(self):
        # Parent tidak raise exception, tapi subclass raise exception
        raise NotImplementedError("ReportOnlyApp tidak mendukung tampilan tabel GUI.")
```

Langkah Perbaikan (Refactoring)

Perbaikan kelas `ReportOnlyApp` dilakukan setelah Iqbal selesai memisahkan `BaseAppManager` menjadi *interface* yang lebih kecil (`IFormView`, `IDataWriter`, dan `IReportable`).

Solusi ISP

- Mengubah Inheritance: ReportOnlyApp tidak lagi mewarisi kelas Application secara keseluruhan, melainkan hanya mewarisi *interface* spesifik yang dibutuhkannya, yaitu IFormView dan IReportable.
- Menghapus Dummy Method: Karena pewarisannya sudah spesifik, *method* save_data() dan delete_data() yang tidak relevan telah dihapus sepenuhnya dari kelas ini.

Solusi LSP

- Mematuhi Kontrak Parent: *Method* showdetails() dikembalikan fungsinya agar tidak menghasilkan *exception*. Perbaikan dilakukan dengan mendelegasikan pemanggilan ke *parent* menggunakan super().showdetails() agar GUI tetap tampil normal.
- Menambahkan Method Baru: Fitur khusus untuk laporan (misalnya cetak di *console*) dipisahkan ke dalam *method* baru yaitu generate_report(), sesuai dengan *interface* IReportable yang baru.

Kode setelah Perbaikan

```
# ReportOnlyApp kini hanya bergantung pada interface yang relevan
# (ISP Terpenuhi)
class ReportOnlyApp(IFormView, IReportable):

    def showdetails(self):
        # Mematuhi kontrak parent: menampilkan GUI tanpa raise
        # exception (LSP Terpenuhi)
        super().showdetails()

    def generate_report(self):
        # Logika khusus pelaporan dipindah ke method yang benar
        print("=== LAPORAN APPOINTMENTS ===")
        for row in self.alldata:
            print(row)
```

Setelah refactoring diterapkan, pengujian berikut telah dilakukan untuk memastikan aplikasi berjalan tanpa merubah *output* akhir:

1. Validasi ISP: Struktur kelas ReportOnlyApp sudah bersih dari *dummy method* (save_data dan delete_data).
2. Validasi LSP: Saat objek diinisiasi dengan app = ReportOnlyApp(window) dan app.showdetails() dipanggil, program tidak lagi *crash* atau

memunculkan `NotImplementedError`, melainkan sukses menampilkan GUI secara normal.